

LangGraph Integration — Routing Between Chat, Retrieval & Prediction

Scenario

The objective of this project was to integrate the AFL (Australian Football League) chatbot developed in Day 3 with the prediction models developed in Day 2 by using **LangGraph**.

The system was designed to understand different types of user queries and route them to the appropriate workflow. The main supported query categories were:

- **Factual Questions** — general AFL rules and information.
- **Retrieval Questions** — retrieving AFL statistics and historical match information from the available dataset.
- **Prediction Questions** — predicting the likely winner of a match or the likely top player using the prediction models.
- **Off-topic Questions** — questions that are unrelated to the AFL system.

The purpose of using LangGraph was to create an explicit and controlled workflow instead of allowing a single general-purpose agent to decide everything by itself. This makes the system easier to validate, trace, debug, and control, particularly for probabilistic predictions.

The final system contains routing, retrieval, prediction, validation, clarification, fallback, and response-formatting components.

Task 1 --- State Schema and Graph Design

Objective

The first task was to design the state structure and overall LangGraph workflow.

A state schema was created to store all important information during execution. The state includes:

- User query
- Conversation history
- Detected intent
- Tool results
- Final response
- Error information

- Clarification status
- Clarification question
- Execution trace

The graph was designed around an explicit router. The router identifies the user's intent and directs the request toward the appropriate branch.

The main workflow consists of:

User Query → Intent Router → Appropriate Branch → Validation → Response Formatting → Final Response

The major branches are:

1. Factual answer
2. Retrieval
3. Prediction
4. Off-topic/fallback

An explicit routing approach was selected instead of relying completely on a generic agent. This provides better control over which tools can be used for different types of requests.

This is especially important for prediction queries because predictions are probabilistic and should not be presented as guaranteed outcomes.

Implementation

A TypedDict-based state schema was created using Python. LangGraph's StateGraph, START, and END components were then used to define the workflow.

An intent classification function was also created to identify factual, retrieval, prediction, and off-topic queries.

Several sample queries were tested to verify the router, including questions about AFL rules, statistics, and match predictions.

Conclusion :

Task 1 successfully established the foundation of the AFL LangGraph application. A structured state schema and explicit graph design were created. The design provides clear separation between factual answers, retrieval, predictions, and unsupported requests, making the overall system more controlled and reliable.

Task 2 --- Build and Test the Intent Router

Objective

The objective of Task 2 was to build an intent router capable of classifying user queries into the correct categories.

The router was designed to identify four main intents:

- **Factual**
- **Retrieval**
- **Prediction**
- **Off-topic**

Keyword and pattern-based rules were used to identify the intent of a query.

Prediction-related words such as:

- predict
- prediction
- win
- winner
- top-score

were used to identify prediction requests.

Retrieval-related words such as:

- stats
- statistics
- record
- history
- last round

were used to identify requests requiring dataset retrieval.

General AFL information and rules were classified as factual questions.

Questions unrelated to AFL were routed to the off-topic/fallback path.

Testing

A variety of AFL and non-AFL questions were tested. Examples included:

- "Who will win Collingwood vs Geelong?"
- "What were Geelong's stats last round?"
- "What are the basic rules of AFL?"
- "Who won the latest NBA game?"
- "Who will top-score for Richmond?"

The router output was compared with the expected intent. Misroutes could then be identified and the routing rules adjusted.

Conclusion :

Task 2 successfully implemented and tested the intent router. The router provides the first decision point in the LangGraph workflow and allows different types of queries to be processed by the appropriate branch. Testing also helped identify and correct potential misrouting cases.

Task 3 --- Integrate Prediction Models as LangGraph Tools

Objective

The purpose of Task 3 was to connect the prediction models from Day 2 with the LangGraph workflow.

The prediction functionality was integrated through prediction tools such as:

- `predict_match_winner`
- `predict_top_player`

The system was also designed to resolve common AFL team aliases.

For example:

- **Pies → Collingwood Magpies**
- **Cats → Geelong Cats**
- **Tigers → Richmond Tigers**
- **Blues → Carlton Blues**
- **Bulldogs → W. Bulldogs**

This allows users to use common team names rather than requiring exact dataset names.

Prediction Workflow

When a prediction query is received, the system first extracts the teams from the query.

For example:

"Who will win Pies vs Cats?"

The system resolves:

Pies → Collingwood Magpies

Cats → Geelong Cats

The resolved team names are then passed to the prediction model.

If both teams cannot be identified, the system does not guess. Instead, it requests clarification from the user.

Prediction responses were also designed to include:

- Predicted outcome
- Probability/confidence information
- 2–3 grounding features
- A statement that the prediction is model-based and not guaranteed

This ensures that the prediction is presented as a probabilistic model output rather than a certain future result.

Dataset Compatibility

During integration, the dataset column names were checked and corrected. The team information was retrieved using the actual `team_name` column rather than an incorrect `team` column.

The match record functionality was also updated to work with the available fields such as:

- `team_name`
- `opponent`
- `team_score`
- `opponent_score`

Conclusion

Task 3 successfully connected the Day 2 prediction models with the LangGraph workflow. Team aliases were resolved, prediction inputs were validated, and prediction responses were designed

to provide probability/confidence information and supporting features. The system also avoids guessing when required team information is missing.

Task 4 --- Self-Correction and Fallbacks

Objective

The objective of Task 4 was to make the system safer and more reliable by adding validation, clarification, and fallback mechanisms.

The system checks whether a prediction or retrieval operation actually produced a usable result.

The validation stage checks:

- Whether an error occurred
- Whether a tool result was returned
- Whether required information is available
- Whether clarification is required

Clarification Handling

If a required team or player cannot be resolved, the system does not invent an answer.

For example:

"Will they win this week?"

This query does not specify the two teams.

Instead of guessing, the system responds with a clarification request such as:

"I need both AFL teams to make the prediction. Which two teams should I compare?"

This prevents incorrect predictions caused by missing information.

Fallback Handling

A fallback node was also created for unsupported or ambiguous requests.

For example:

"Predict the exact weather during the AFL match."

This is outside the supported capabilities of the AFL prediction system.

The fallback response explains that the system supports AFL factual questions, statistics, and supported predictions rather than attempting to fabricate an answer.

Self-Correction

The workflow records the validation and clarification steps in the state trace.

The general logic is:

Tool → Validation →

- Valid result → Response Formatting
- Missing information → Clarification
- Error/unsupported request → Fallback

This improves reliability and reduces the possibility of hallucinated results.

Conclusion

Task 4 successfully added validation, self-correction, clarification, and fallback handling. The system now avoids guessing when required information is missing and provides controlled responses for unsupported requests. These mechanisms improve the reliability and safety of the overall AFL application.

Task 5 --- End-to-End Testing

Objective

The objective of Task 5 was to test the complete LangGraph workflow from the initial user query to the final response.

The testing covered different paths of the application, including:

- Factual questions
- Retrieval questions
- Prediction questions
- Ambiguous questions
- Off-topic/unsupported questions

End-to-End Test Cases

Ten different queries were prepared for testing.

Test 1 — Factual

Query:

"What are the basic rules of AFL?"

Expected Intent:

Factual

Test 2 — Factual

Query:

"What is a goal in AFL?"

Expected Intent:

Factual

Test 3 — Retrieval

Query:

"What were Geelong's stats last round?"

Expected Intent:

Retrieval

Test 4 — Retrieval

Query:

"Show me the record between Collingwood and Geelong."

Expected Intent:

Retrieval

Test 5 — Prediction

Query:

"Who will win Collingwood vs Geelong?"

Expected Intent:

Prediction

Test 6 — Prediction with Aliases

Query:

"Who will win Pies vs Cats?"

Expected Intent:

Prediction

The system resolves Pies and Cats to their official dataset names before prediction.

Test 7 — Player Prediction

Query:

"Who will top-score for Richmond?"

Expected Intent:

Prediction

Test 8 — Off-topic**Query:**

"Who won the latest NBA game?"

Expected Intent:

Off-topic

The system should not attempt to answer a non-AFL sports question using the AFL tools.

Test 9 — Unsupported Request**Query:**

"Predict the exact weather during the AFL match."

Expected Intent:

Off-topic/Unsupported

The fallback mechanism handles this request.

Test 10 — Ambiguous Prediction**Query:**

"Will they win this week?"

Expected Handling:

Clarification

The system should ask which teams the user is referring to instead of guessing.

State Trace Testing

Three representative state traces were examined.

Trace 1 — Prediction Path

The prediction workflow follows the general path:

User Query → Intent Router → Prediction → Validation → Response Formatting → Final Response

The trace records the detected intent, prediction result, validation status, and final response.

Trace 2 — Retrieval Path

The retrieval workflow follows:

User Query → Intent Router → Retrieval → Validation → Response Formatting → Final Response

The state trace makes it possible to inspect the retrieved information and verify that a result was returned.

Trace 3 — Clarification Path

An ambiguous query such as:

"Will they win this week?"

follows:

User Query → Intent Router → Prediction Input Validation → Clarification → End

The system requests the missing team information rather than making an unsupported assumption.

LangGraph vs Monolithic LangChain Agent

The LangGraph implementation uses explicit nodes and conditional transitions. This makes the workflow easier to understand and trace.

A monolithic LangChain agent would generally rely on one agent to decide which tool to use and how to handle the complete request.

The LangGraph approach provides explicit control over:

- Intent routing
- Tool selection
- Prediction validation
- Retrieval validation
- Clarification
- Fallback handling
- State tracing

The main trade-off is that LangGraph requires more implementation code because the workflow and transitions are explicitly defined.

Conclusion

Task 5 completed the end-to-end testing of the AFL LangGraph application. Multiple query types and workflow paths were tested, including factual, retrieval, prediction, clarification, and unsupported requests.

The state traces demonstrated how queries move through the graph and how validation and fallback mechanisms control the final response.

Overall Project Conclusion

The project successfully integrated the Day 2 AFL prediction models and Day 3 AFL chatbot/retrieval functionality using LangGraph.

The completed system contains an explicit intent-routing mechanism that directs user queries to the appropriate workflow. Factual questions are handled through the factual path, statistics and historical information are handled through retrieval, and prediction requests are processed through the integrated prediction models.

The system also supports common AFL team aliases, validates prediction inputs, provides probabilistic prediction information, and avoids presenting predictions as guaranteed outcomes.

Validation, clarification, and fallback mechanisms were added to prevent the system from guessing when information is missing or when a request is outside the supported capabilities.

Finally, end-to-end testing was performed across multiple query types, and state traces were examined to verify the workflow.

Overall, the implementation demonstrates how LangGraph can be used to build a controlled, traceable, and modular AFL conversational application that combines chat, retrieval, and machine-learning prediction capabilities within a single workflow.